

Interaktivní nástroj pro vizualizaci algoritmů a datových struktur

Interactive Tool for Visualizing Algorithms and Data Structures

Barbora Kovalská

Diplomová práce

Vedoucí práce: Ing. Radoslav Fasuga, Ph.D.

Ostrava, 2026

Zadání diplomové práce

Student:

Bc. Barbora Kovalská

Studijní program:

N0613A140034 Informatika

Téma:

Interaktivní nástroj pro vizualizaci algoritmů a datových struktur
Interactive Tool for Visualizing Algorithms and Data Structures

Jazyk vypracování:

čeština

Zásady pro vypracování:

Cílem projektu je vytvořit webový nástroj pro vizualizaci datových struktur (např. zásobník, fronta, seznamy, stromy) a simulaci algoritmů (třídící a vyhledávací algoritmy). Aplikace bude sloužit jako výukový prostředek, umožňující interaktivní operace a teoretické vysvětlení jednotlivých datových struktur a algoritmů.

1. Student se seznámí s problematikou datových struktur a základních algoritmů.
2. Student provede analýzu existujících výukových nástrojů zaměřených na vizualizaci algoritmů a navrhne vlastní specifikace pro webovou aplikaci.
3. Student navrhne a implementuje interaktivní webovou aplikaci s přívětivým uživatelským rozhraním. Aplikace bude obsahovat moduly pro vizualizaci základních datových struktur a algoritmů, interaktivní simulaci operací nad strukturami s možností sledování v reálném čase a také teoretické vysvětlení daných postupů.
4. Výsledkem bude plně funkční prototyp webové aplikace a dokumentace popisující použití a implementaci nástroje.
5. Srovnávejte výslednou aplikaci s existujícími aplikacemi.

Seznam doporučené odborné literatury:

- [1] LEVITIN, Anany. Introduction to the design and analysis of algorithms. Third edition. Boston: Pearson, c2012. ISBN 978-0132316811.
- [2] CORMEN, Thomas H.; LEISERSON, Charles Eric; RIVEST, Ronald L. a STEIN, Clifford. Introduction to algorithms. Fourth edition. Cambridge, Massachusetts: The MIT Press, [2022]. ISBN 9780262046305.
- [3] SEDGEWICK, Robert a WAYNE, Kevin Daniel. **Algorithms**. Fourth edition. Boston: Addison-Wesley, [2011]. ISBN 978-0-321-57351-3
- [4] Gonzalez, V. (2020). JavaScript Data Structures and Algorithms: An Introduction to Solving Complex Problems Using JavaScript. Packt Publishing.
- [5] Bamford, K., & Ruiz, D. (2022). Frontend Architecture for Design Systems: A Modern Guide to UI Development. O'Reilly Media.

Formální náležitosti a rozsah diplomové práce stanoví pokyny pro vypracování zveřejněné na webových stránkách fakulty.

Vedoucí diplomové práce: **Ing. Radoslav Fasuga, Ph.D.**

Datum zadání: 01.09.2025

Datum odevzdání: 30.04.2026

Garant studijního programu: prof. RNDr. Václav Snášel, CSc.

V IS EDISON zadáno: 09.10.2025 14:34:54

Abstrakt

Abstraktní čeština.

Klíčová slova

interaktivní nástroj; vizualizace; algoritmy; datové struktury; vzdělávání; simulace

Abstract

Abstract English.

Keywords

interactive tool; visualization; algorithms; data structures; education; simulation

Poděkování

Děkuji, že mám ještě tolik času na dokončení této práce.

Obsah

Seznam použitých symbolů a zkratk	8
Seznam obrázků	9
Seznam tabulek	10
Seznam výpisů zdrojového kódu	11
1 Úvod	12
2 Teorie ohledně obsahu aplikace	13
2.1 Datové struktury	13
2.1.1 Lineární datové struktury	13
2.1.2 Nelineární (stromové) datové struktury	15
2.1.3 Prioritní fronty a haldy	22
2.1.4 Srovnání popsaných datových struktur	24
2.2 Třídící algoritmy	25
2.2.1 Algoritmy s kvadratickou časovou složitostí	25
2.2.2 Algoritmy s logaritmickou časovou složitostí	28
2.2.3 Srovnání třídících algoritmů	32
3 Analýza existujících řešení	34
3.1 Vizualizace datových struktur	34
3.1.1 Gnarley Trees	34
3.1.2 Data Structure Visualizations	35
3.1.3 B Plus Tree	36
3.1.4 Porovnání	36
3.2 Simulace třídících algoritmů	37
3.2.1 Sort Visualizer	38
3.2.2 VisuAlgo	38

3.2.3	Toptal Sorting Algorithms Animations	39
3.2.4	Porovnání	40
3.3	Shrnutí	41
4	Požadavky na výukovou aplikaci	42
4.1	Funkcionalita	42
4.2	Použitelnost	43
4.3	Spolehlivost	43
4.4	Výkonnost	44
4.5	Podpořitelnost a udržitelnost	44
5	Analýza a volba použitých technologií	45
5.1	Architektura aplikace	45
5.1.1	Client-Server a Jednostránková webová aplikace	45
5.1.2	Strategie renderování	46
5.2	Webový framework	46
5.3	Grafová knihovna	46
5.4	CI/CD	46
5.5	Shrnutí	46
6	Závěr	47
	Literatura	48
	Přílohy	49

Seznam použitých zkratek a symbolů

VŠB	– Vysoká škola Báňská
LIFO	– Last In, First Out
FIFO	– First In, First Out
BST	– Binární vyhledávací strom
HTTP	– Hypertext Transfer Protocol
SPA	– Single Page Application
API	– Application Programming Interface
AJAX	– Asynchronous JavaScript and XML
JSON	– JavaScript Object Notation
SEO	– Search Engine Optimization

Seznam obrázků

2.1	Pole a spojový seznam s n prvky	14
2.2	Zásobník a fronta	14
2.3	Binární vyhledávací strom	15
2.4	AVL strom	16
2.5	Pravá (R) a dvojitá pravá-levá (RL) rotace AVL stromu	17
2.6	Červeno-černý strom	18
2.7	Oprava červeno-černého stromu po vložení prvku, případ 1	19
2.8	Oprava červeno-černého stromu po vložení prvku, případ 2	19
2.9	B-strom řádu 3	20
2.10	Vložení klíče do B-stromu řádu 3	21
2.11	Binární halda reprezentovaná stromem	23
2.12	Binární halda reprezentovaná polem	23
5.1	Srovnání životních cyklů stránek v klasické client-server architektuře a v SPA	46

Seznam tabulek

2.1	Srovnání vlastností a časové složitosti operací datových struktur	24
2.2	Srovnání vlastností a složitosti třídících algoritmů	33
3.1	Porovnání analyzovaných aplikací pro vizualizaci datových struktur	37
3.2	Podporované datové struktury v analyzovaných aplikacích	37
3.3	Porovnání analyzovaných aplikací pro vizualizaci třídících algoritmů	40

Seznam výpisů zdrojového kódu

1	Pseudokód pomocné funkce pro výměnu dvou prvků	25
2	Pseudokód algoritmu Bubble Sort	26
3	Pseudokód algoritmu Selection Sort	27
4	Pseudokód algoritmu Insertion Sort	28
5	Pseudokód algoritmu Merge Sort	29
6	Pseudokód algoritmu Quick Sort	30
7	Pseudokód algoritmu Heap Sort	32

Kapitola 1

Úvod

Vítejte v mé magisterské práci.

Kapitola 2

Teorie ohledně obsahu aplikace

V této kapitole jsou popsány teoretické základy týkající se obsahu aplikace. Nejprve je uveden přehled různých datových struktur, které aplikace podporuje, včetně jejich vlastností, operací a požadavků na vizualizaci. Dále jsou diskutovány třídící algoritmy, které budou implementovány, a jejich význam pro pochopení základních konceptů informatiky. Nakonec je popsán přístup k vizualizaci těchto datových struktur a algoritmů, včetně použitých technik a metod pro efektivní zobrazení a interakci s uživatelem.

2.1 Datové struktury

V této kapitole jsou popsány datové struktury, které jsou podstatné pro oblast informatiky. Každá datová struktura je charakterizována svými vlastnostmi, operacemi a požadavky na vizualizaci.

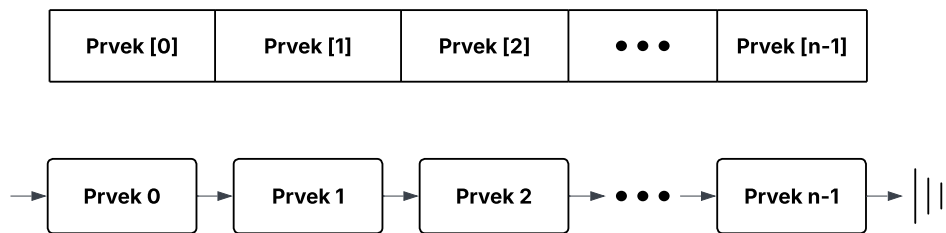
2.1.1 Lineární datové struktury

Pro účely výuky datových struktur je rozumné začít s lineárními datovými strukturami, které jsou jednodušší na pochopení a vizualizaci. Tyto struktury zahrnují mimo jiné pole, spojové seznamy, zásobníky a fronty.

I přes to, že tyto struktury jsou relativně jednoduché a již moc dobře známé, v této práci budou teoreticky uvedeny, aby byla zajištěna úplnost a konzistence s dalšími částmi práce.

Pole a spojový seznam

Základním stavebním kamenem lineárních datových struktur jsou pole a spojové seznamy. Pole (*Array*) poskytují rychlý přístup k prvkům pomocí indexů, zatímco spojové seznamy (*Linked List*) umožňují dynamickou alokaci paměti a efektivní vkládání a mazání prvků pomocí ukazatelů. Obrázek 2.1 znázorňuje rozdíl mezi těmito dvěma strukturami. [1]



Obrázek 2.1: Pole a spojový seznam s n prvky

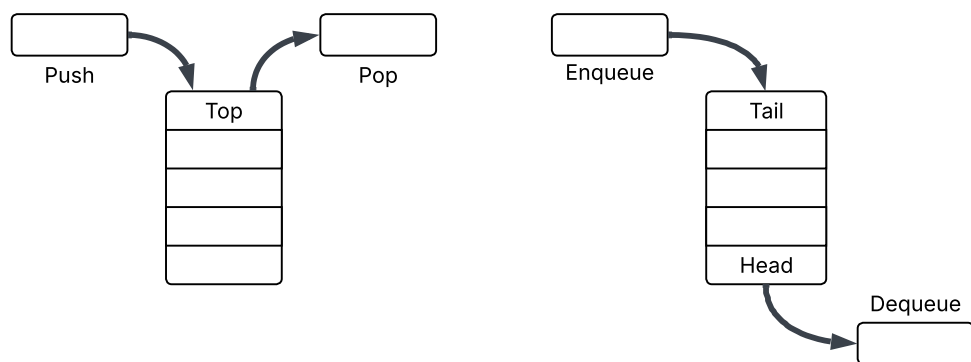
Spojový seznam je tvořen uzly, kde každý uzel obsahuje datovou část a ukazatel na další uzel v seznamu. V případě obousměrného spojového seznamu obsahuje každý uzel také ukazatel na předchozí uzel, což umožňuje obousměrnou navigaci. List musí mít ukazatel na první uzel (*head*) a často také na poslední uzel (*tail*) pro efektivní přidávání prvků na konec seznamu. [2]

Spojový seznam i pole se využívají pro implementaci abstraktnějších seznamů, které obsahují sekvenci prvků a poskytují operace pro přidávání, odebrání a přístup k prvkům. [1]

Zásobník a fronta

Speciálním případem lineárních seznamů jsou zásobníky (*Stack*) a fronty (*Queue*), jejichž vizualizaci je možné vidět na Obrázku 2.2. Jejich hlavní charakteristikou je předem specifikovaný způsob mazání a přidávání prvků.

Zásobník funguje na principu LIFO (*Last In, First Out*), což znamená, že poslední prvek přidaný do zásobníku je prvním, který je z něj odebrán. Operace přidání prvku se běžně nazývá *push* a operace odebrání prvku se nazývá *pop*. V zásobníku je přístup k prvkům omezen na vrchol zásobníku, jež se nazývá *top*. [2]



Obrázek 2.2: Zásobník a fronta

Naopak fronta funguje na principu FIFO (*First In, First Out*), kde první prvek přidaný do fronty je prvním, který je z ní odebrán. Operace přidání prvku se nazývá *enqueue* a operace odebrání prvku se nazývá *dequeue*. Fronta se implementuje pomocí ukazatele, který sleduje začátek (*head*) fronty, a často bývá doplněna o ukazatel na konec (*tail*) fronty, což umožňuje efektivní přidávání a odebírání prvků. [2]

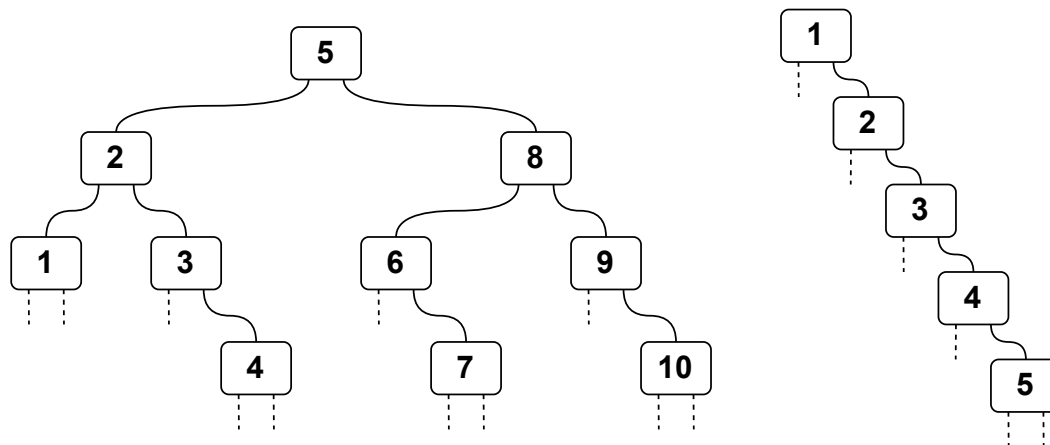
Obě struktury mohou být efektivně implementovány pomocí spojových seznamů nebo polí, v závislosti na požadavcích na výkon a paměťovou efektivitu. [1]

2.1.2 Nelineární (stromové) datové struktury

Mezi nelineární datové struktury patří stromy, hierarchické struktury představující spojitý acyklický graf, kde každý prvek (uzel) může mít více potomků. Stromy obecně podporují operace **vkládání**, **mazání** a **vyhledávání** prvků, přičemž složitost těchto operací závisí na konkrétním typu stromu a jeho struktuře. Tyto struktury se často používají pro reprezentaci hierarchických dat, jako jsou souborové systémy, organizační struktury nebo různé druhy indexů v databázích. V následujících podkapitolách jsou popsány různé typy stromů, které budou implementovány v aplikaci, včetně jejich vlastností, operací a požadavků na vizualizaci.

Binární vyhledávací strom

Binární vyhledávací stromy (*Binary Search Trees, BST*) jsou speciálním typem binárních stromů, kde každý uzel splňuje následující vlastnost: klíče v levém podstromu jsou menší než klíč v uzlu, a klíče v pravém podstromu jsou větší než klíč v uzlu. Tato vlastnost umožňuje efektivní vyhledávání, vkládání a mazání prvků ve stromu. Příklad binárního vyhledávacího stromu je znázorněn na Obrázku 2.3. [2]

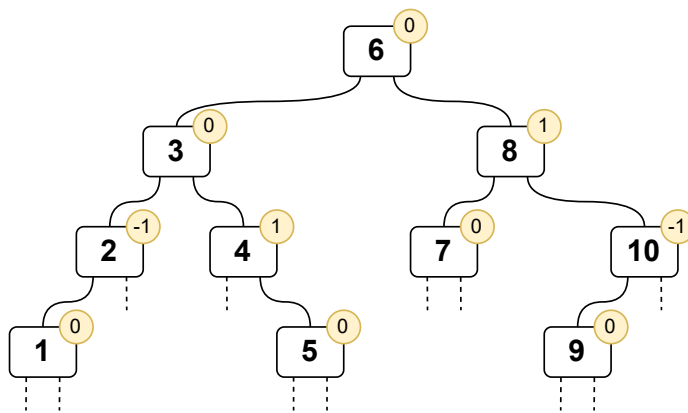


Obrázek 2.3: Binární vyhledávací strom

Efektivita operací nad BST závisí na jeho výšce. Zde nastává problém degradace BST, kdy se strom stává nevyváženým a jeho výška se blíží počtu uzlů, což vede k lineární složitosti operací. V nejhorším případě se BST může stát podobným spojovému seznamu, což výrazně snižuje jeho výkon. Degradovaný strom je možné vidět na Obrázku 2.3. I přes to, že tedy operace nad BST mají průměrnou složitost $O(\log n)$, v nejhorším případě může být složitost $O(n)$. Pro zefektivnění těchto operací se používají samovyvažující se stromy, které z binárního vyhledávacího stromu vychází. [1]

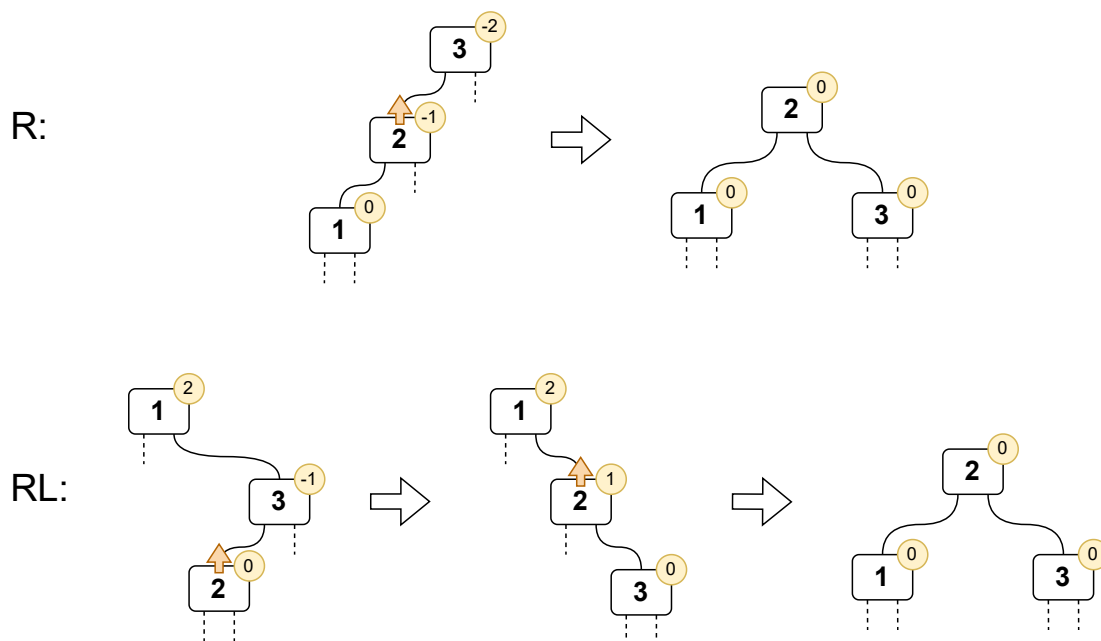
AVL strom

Samovyvažující se AVL stromy jsou pojmenovány po svých tvůrcích, G. M. Adelson-Velsky a E. M. Landis, kteří je představili v roce 1962. AVL strom je binární vyhledávací strom, který udržuje vyváženost tím, že zajišťuje, aby výškový rozdíl mezi levým a pravým podstromem každého uzlu byl nejvýše 1. Tato vlastnost jde vidět na Obrázku 2.4, kde má každý uzel přidání informaci o jeho vyvážení, znázorněnou jako číslo ve žlutém kruhu, které pro tento typ stromu může nabývat pouze hodnot -1, 0 nebo 1. Vyvážení je definováno jako rozdíl mezi výškou levého a pravého podstromu. V samotné implementaci se často v každém uzlu ukládá právě výška podstromu namísto vyvážení, což umožňuje efektivní výpočet vyvážení během operací vkládání a mazání. AVL stromy zajišťují, že všechny operace mají složitost $O(\log n)$, což je důvod, proč jsou často používány v situacích, kde je důležitá rychlost přístupu k datům. [1, 2]



Obrázek 2.4: AVL strom

Vyvážení AVL stromů se udržuje pomocí rotací, které jsou prováděny po každé operaci vkládání nebo mazání, pokud dojde k narušení vyváženosti. Rotace představuje lokální transformaci podstromu právě toho vrcholu, kde je podmínka vyváženosti narušena (kde nabyla hodnoty -2 nebo 2). Existují čtyři typy rotací: jednoduchá pravá rotace, jednoduchá levá rotace, dvojitá levá-pravá a dvojitá pravá-levá rotace, přičemž levé a pravé varianty jsou zrcadlové obrazy. [1]



Obrázek 2.5: Pravá (R) a dvojitá pravá-levá (RL) rotace AVL stromu

Názornou ukázkou pravých variant rotací (R a RL) je možné vidět na Obrázku 2.5. Během rotace se mění struktura podstromu, což umožňuje obnovit vyváženost stromu a zajistit, že všechny operace zůstanou v logaritmické složitosti. Například **pravá rotace** se provádí, když levý podstrom vrcholu je příliš vysoký, a zahrnuje přesunutí levého potomka vrcholu nahoru a samotného vrcholu dolů do pravého podstromu. Akce přesunutí potomka nahoru je znázorněna oranžovou šipkou nad daným vrcholem. **Dvojitá pravá-levá rotace** se provádí, když pravý podstrom vrcholu je příliš vysoký a zároveň má levý podstrom, který je vyšší než jeho pravý podstrom. Tato rotace zahrnuje nejprve pravou rotaci na pravém potomku vrcholu a poté levou rotaci na vrcholu samotném. [1]

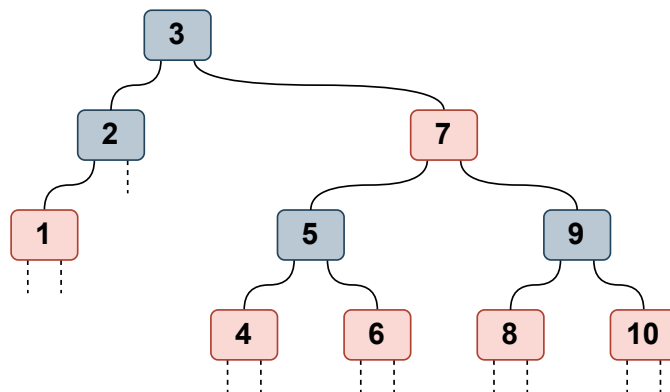
Tyto rotace sice zvyšují složitost implementace AVL stromů, ale zajišťují, že všechny operace nad stromem mají složitost $\Theta(\log n)$. Kvůli počtu rotací, které mohou být nutné po každé operaci vkládání nebo mazání, sice zabránily AVL stromům stát se standardem pro implementaci seznamů¹, ale jejich myšlenka a principy jsou základem pro další samovyvažující se stromy, které z nich vycházejí. [1]

Červeno-černý strom

Dalším typem samovyvažujícího se stromu je červeno-černý strom, pojmenován podle barvy, kterou jsou označeny jeho uzly. Tento strom udržuje vyváženost pomocí pravidel týkajících se barev uzlů, což zajišťuje, že žádná cesta od kořene k listu není více než dvakrát delší než jakákoli jiná

¹Seznam je v tomto kontextu myšlen jako datová struktura anglicky nazvaná *dictionary*.

cesta. Červeno-černé stromy jsou často používány v implementacích map a množin, protože poskytují dobrý kompromis mezi složitostí implementace a výkonem. Příklad červeno-černého stromu je znázorněn na Obrázku 2.6. [2]



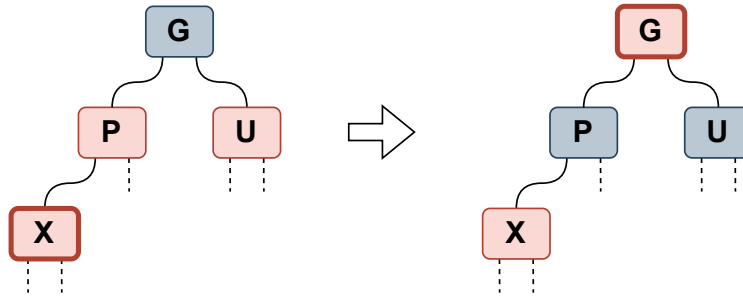
Obrázek 2.6: Červeno-černý strom

Červeno-černé stromy jsou binární vyhledávací stromy, které musí splňovat následující pravidla, aby byla zajištěna jejich vyváženost:

1. Každý uzel je buď červený, nebo černý.
2. Kořen stromu je vždy černý.
3. Všichni nezaplňení potomci listů jsou černí.
4. Pokud je uzel červený, oba jeho potomci jsou černí.
5. Každá cesta od daného uzlu k jeho potomkům obsahuje stejný počet černých uzlů.

Dodržování těchto pravidel zajišťuje, že strom zůstává vyvážený, což umožňuje efektivní operace vkládání, mazání a vyhledávání s logaritmickou složitostí $\Theta(\log n)$. Implementace červeno-černých stromů zahrnuje rotace a změny barev uzlů, které jsou prováděny po každé operaci vkládání nebo mazání, aby se udržela vyváženost stromu. Rotace jsou podobné těm, které se používají v AVL stromech, ale pravidla pro změnu barev uzlů jsou specifická pro červeno-černé stromy a zajišťují, že strom zůstává vyvážený i v případě, že dojde k narušení pravidel během operací. [2]

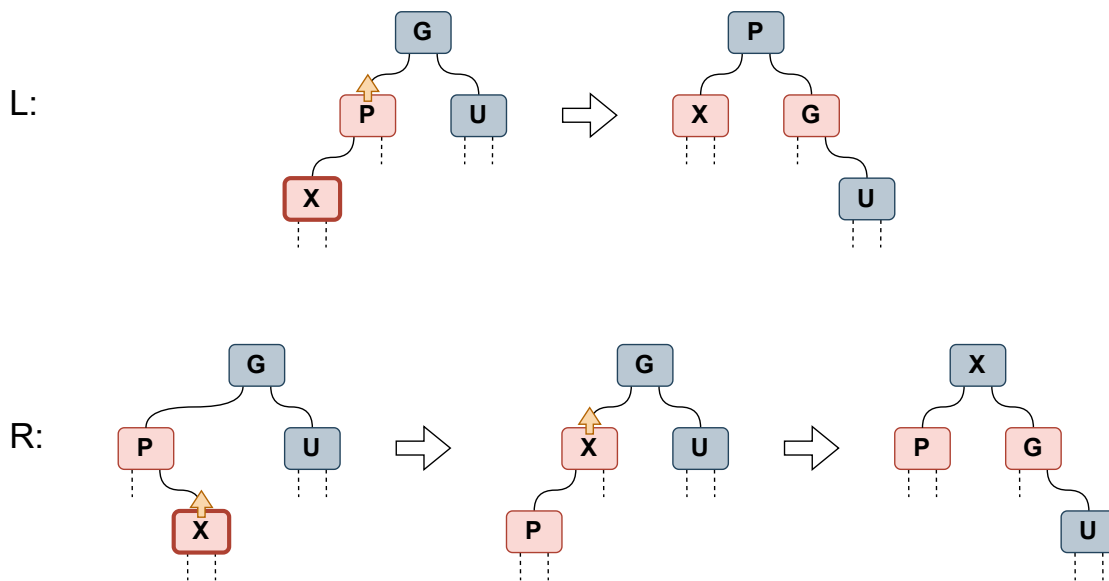
Jako příklad operace červeno-černého stromu bude popsána implementace **vložení prvku**. Při vkládání nového prvku do červeno-černého stromu se nový uzel nejprve vloží jako červený, aby se minimalizovalo narušení vyváženosti stromu. Poté se provádí kontrola a případné opravy, které zahrnují rotace a změny barev uzlů, aby se zajistilo dodržení pravidel červeno-černého stromu. Pro ilustraci této operace je použit podstrom, kde jsou zobrazeny uzly důležité pro opravu stromu.



Obrázek 2.7: Oprava červeno-černého stromu po vložení prvku, případ 1

Jedná se o čtyři uzly: nově přidaný uzel (**X**), jeho rodič (**P**), strýc (**U**) a prarodič (**G**). Konkrétní typ opravy se provádí v závislosti na barvě strýce (**U**) a pozici nově přidaného uzlu (**X**). [3]

První případ nastává, když **strýc (U) je červený**, jak je vidět na Obrázku 2.7. V tomto případě se provádí změna barev: **P** a **U** se zbarví na černou, zatímco **G** se zbarví na červenou. Poté se pokračuje v kontrole a případných opravách prarodiče **G**, tato zpětná kontrola je znázorněna tlustým okrajem kolem daného uzlu. Rekurzivně se tedy strom projde až ke kořenu, aby se zajistilo, že všechna pravidla červeno-černého stromu jsou dodržena. Tento proces může vést k dalším změnám barev a rotacím, pokud dojde k narušení pravidel během opravy. [3]



Obrázek 2.8: Oprava červeno-černého stromu po vložení prvku, případ 2

Druhým případem je situace, kdy **strýc (U) je černý**, jak je znázorněno na Obrázku 2.8. V tomto případě se provádí nejen změna barev, ale i rotace. Dále se situace větví podle pozice nově přidaného uzlu (**X**) a jeho rodiče (**P**). Pro zjednodušení jsou uvedeny pouze dvě varianty, kdy je **P**

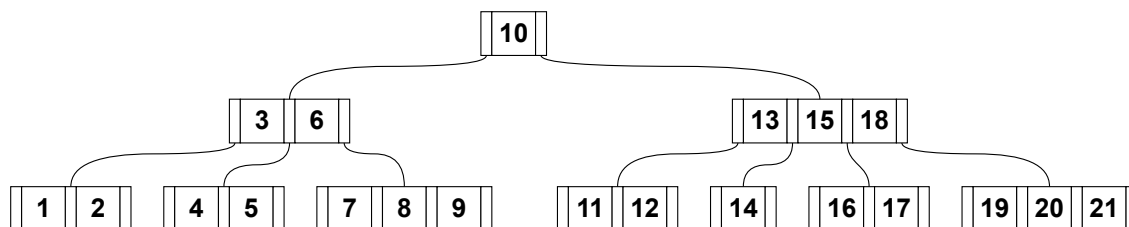
levým potomkem G . Pokud je nový uzel **levým potomkem** (případ L), provede se pravá rotace na G a P se tedy stává novým kořenem podstromu, přičemž během rotace si zmíněné vrcholy vymění barvy. Podobně jako u diagramu AVL stromu, akce přesunutí potomka směrem nahoru je na diagramu znázorněna oranžovou šipkou nad daným vrcholem. Pokud je nový uzel **pravým potomkem** (případ R), provede se nejprve levá rotace na P , čímž se X stává levým potomkem G , a poté pravá rotace na G , přičemž během druhé rotace si X a G vymění barvy. Po těchto úpravách již není potřeba rekursivní kontrola, protože všechna pravidla červeno-černého stromu jsou dodržena a strom zůstává vyvážený. [3]

Odstraňování prvku z červeno-černého stromu je složitější operace, která zahrnuje více případů a může vyžadovat několik rotací a změn barev, aby se udržela vyváženost stromu. Nicméně principy jsou podobné jako při vkládání, kdy se nejprve odstraní uzel a poté se provádí opravy, pokud dojde k narušení pravidel červeno-černého stromu. [2]

B-stromy

Posledním typem datové struktury z oblasti stromů, který bude implementován v aplikaci, jsou B-stromy. B-stromy, které představili R. Bayer a E. McGreight, jsou samovyvažující se stromy navržené pro efektivní ukládání a vyhledávání dat na sekundárních úložištích, jako jsou pevné disky. V praxi je možné tyto stromy vidět primárně jako implementaci databázových indexů. B-stromy umožňují ukládání více klíčů v jednom uzlu a mají proměnný počet potomků, což zajišťuje, že strom zůstává vyvážený i při velkém množství dat.

Příklad B-stromu je znázorněn na Obrázku 2.9, kde jde vidět struktura stromu, dost odlišná od předchozích uvedených struktur. Každý uzel může obsahovat více klíčů a mít více potomků, přičemž všechny listy jsou na stejné úrovni, což zajišťuje vyváženost stromu. B-stromy jsou navrženy tak, aby minimalizovaly počet diskových operací potřebných pro vyhledávání, vkládání a mazání dat, což je důvod, proč jsou široce používány v databázových systémech a souborových systémech. [1]



Obrázek 2.9: B-strom řádu 3

B-stromy jsou definovány parametrem m , který se nazývá jeho řádem a může nabývat hodnoty od 2 do nekonečna. Je nutno zmínit, že v jiné literatuře se namísto řádu používá minimální stupeň t , který je roven $\lceil m/2 \rceil$ a udává minimální počet potomků, které musí mít každý uzel kromě kořene a listů. Dále však v tomto textu bude použit pouze řád m . [1, 2]

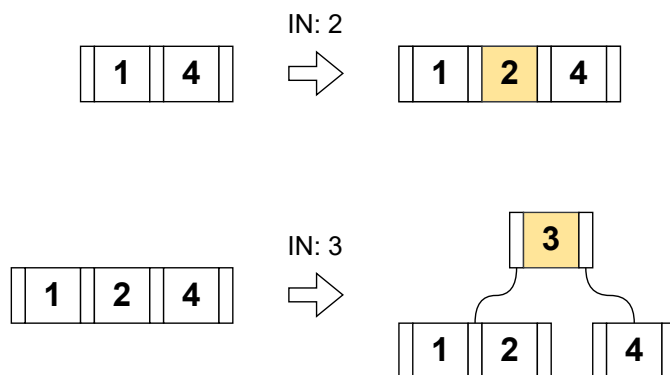
Každý uzel v B-stromu může obsahovat nejvýše $m - 1$ klíčů a mít nejvýše m potomků. Uzel, který není listem, musí mít alespoň $\lceil m/2 \rceil$ potomků, což zajišťuje, že strom zůstává vyvážený. Vztah mezi klíči a potomky v B-stromu je takový, že klíče v uzlu rozdělují rozsah hodnot, které mohou být uloženy v jeho potomcích, při operacích tedy slouží k navigaci stromem. Listy B-stromu jsou všechny na stejné úrovni, což znamená, že všechny cesty od kořene k listům mají stejnou délku. Operace vkládání a mazání v B-stromech zahrnují rozdělení a sloučení uzlů, aby se udržela struktura stromu a jeho vyváženost. [1]

Strom řádu m tedy musí splňovat následující vlastnosti:

1. Kořen stromu je buď list, nebo má 2 až m potomků.
2. Každý uzel kromě kořene a listů musí mít mezi $\lceil m/2 \rceil$ a m potomků.
3. Strom je perfektně vyvážený, což znamená, že všechny listy jsou na stejné úrovni.

Vyhledávání v B-stromu začíná u kořene a porovnává hledanou hodnotu s klíči v uzlu. Pokud se hodnota shoduje s některým z klíčů, operace je úspěšná a hledání končí. Pokud se hodnota neshoduje, klíče se použijí jako hraniční body určující, do kterého potomka se má hledání přesunout. Tento proces se opakuje, dokud se nenajde hledaná hodnota nebo nedojde k listu, což znamená, že hledaná hodnota není ve stromu přítomna. [1]

Operace **vložení** listu do B-stromu je stále přímočará, vyžaduje však případné rozdělení uzlů, pokud se počet klíčů v uzlu překročí. Pokud je uzel plný (obsahuje $m - 1$ klíčů) a je třeba do něj vložit nový klíč, uzel se rozdělí na dva a prostřední klíč se přesune do nadřazeného uzlu. Tento proces může rekurzivně pokračovat až k kořeni, což může vést k vytvoření nového kořene, pokud i ten bude plný. Na Obrázku 2.10 jsou znázorněny oba případy, prvně při vložení klíče 2 do listu, který není plný, a následně při vložení klíče 3, kdy již dochází k rozdělení listu a přesunu klíče 3 do nadřazeného uzlu, který se stává novým kořenem. Nově přidávané klíče jsou na obrázku zvýrazněny oranžovou barvou. [1]



Obrázek 2.10: Vložení klíče do B-stromu řádu 3

Operace **mazání** klíče z B-stromu je složitější, protože může vést k narušení struktury stromu a jeho vyváženosti. Pokud je klíč, který má být odstraněn, v listu, jednoduše se odstraní. Pokud je klíč v uzlu, který není listem, musí být nahrazen buď největším klíčem z levého podstromu (predecessor), nebo nejmenším klíčem z pravého podstromu (successor). Po odstranění klíče může dojít k situaci, kdy uzel obsahuje méně než $\lceil m/2 \rceil - 1$ klíčů, což znamená, že je potřeba provést opravu stromu. Oprava zahrnuje buď přesun klíče z přilehlého sourozeneckého uzlu (pokud má dostatek klíčů), nebo sloučení s přilehlým sourozeneckým uzlem, což může rekurzivně pokračovat až k kořeni. [1]

B-stromy jsou navrženy tak, aby minimalizovaly počet diskových operací potřebných pro vyhledávání, vkládání a mazání dat, což je důvod, proč jsou široce používány v databázových systémech a souborových systémech. Jejich schopnost udržovat vyváženost a efektivně spravovat velké množství dat na sekundárních úložištích je klíčovým faktorem pro jejich popularitu v těchto oblastech a proto je důležité, aby byly zahrnuty do výukové aplikace.

2.1.3 Prioritní fronty a haldy

Prioritní fronty jsou abstraktní datové struktury, které umožňují ukládání prvků s přiřazenou prioritou a poskytují operace pro vkládání prvků a odebírání prvku s nejvyšší prioritou. Oproti běžným frontám, které fungují na principu FIFO, prioritní fronty umožňují flexibilnější způsob správy prvků, což je užitečné v mnoha algoritmech a aplikacích, jako jsou plánování úloh, algoritmy pro hledání nejkratší cesty a další. Halda (*Heap*) je konkrétní implementací prioritní fronty, která využívá stromovou strukturu k efektivnímu ukládání a správě prvků podle jejich priorit. Na závěr teorie datových struktur jsou tedy popsány vlastnosti prioritních front a hald, včetně jejich operací a požadavků na vizualizaci.

Binární halda (Heap)

Binární halda je datová struktura, která může být reprezentována jak polem, tak binárním stromem. V této práci je jako příklad uvedena takzvaná **max-heap**, kde každý rodičovský uzel má hodnotu větší nebo rovnou hodnotám svých potomků, druhá varianta, **min-heap**, se z tohoto popisu dá analogicky odvodit.

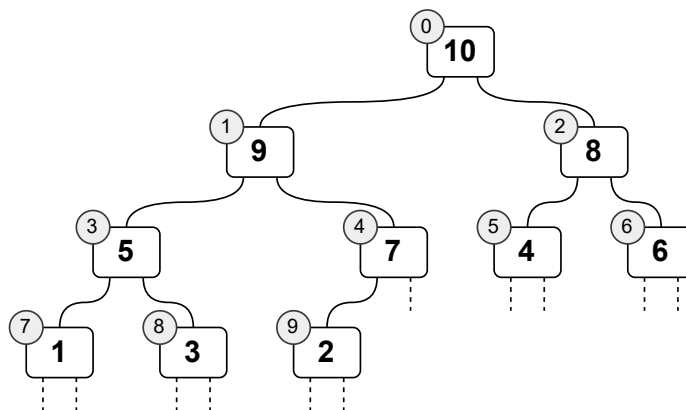
Halda se často používá pro implementaci prioritních front, protože umožňuje efektivní vkládání prvků a nalezení odebírání prvku s nejvyšší prioritou. Díky své struktuře umožňuje binární halda provádět tyto operace s logaritmickou složitostí, je potvrzeno, že v kořenu stromu je vždy ten největší prvek, a dobře se na ni aplikují rekurzivní algoritmy, neboť libovolný podstrom haldy je také haldou. [1]

Jak již bylo zmíněno výše, binární halda reprezentovaná stromem musí splňovat dvě podmínky:

1. Strom musí být kompletní, což znamená, že všechny úrovně stromu jsou zcela zaplněné kromě poslední úrovně, která je zaplněna zleva doprava.

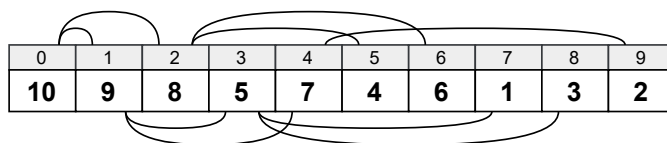
2. Každý rodičovský uzel musí mít hodnotu větší nebo rovnou hodnotám svých potomků.²

Stromovou reprezentaci binární haldy splňující tyto podmínky je možné vidět na Obrázku 2.11. V šedých kruzích u jednotlivých uzlů je znázorněn jejich index, který odpovídá jejich pozici v poli.



Obrázek 2.11: Binární halda reprezentovaná stromem

Binární halda může být také efektivně reprezentována polem (Obrázek 2.12), kde pro každý prvek v poli platí následující vztahy: pro prvek na indexu i jsou jeho levý potomek na indexu $2i$, pravý potomek na indexu $2i + 1$ a rodičovský uzel na indexu $\lfloor i/2 \rfloor$. Tato reprezentace umožňuje efektivní přístup k prvkům haldy, dovoluje algoritmům pracovat s daty bez nutnosti explicitní reprezentace stromové struktury, a je často používána v implementacích algoritmů, jako je Heapsort. [2]



Obrázek 2.12: Binární halda reprezentovaná polem

Základními operacemi nad binární haldou jsou **vložení** a **odebrání** prvku s nejvyšší prioritou. Při vložení nového prvku do haldy se nový prvek umístí na konec stromu (nebo do poslední pozice pole) a poté se provádí operace zvaná *heapify-up*, která zajišťuje, že vlastnost haldy je zachována. Tato operace zahrnuje porovnávání nově vloženého prvku s jeho rodičovským uzlem a případné prohození, pokud je nový prvek větší než jeho rodič. Tento proces se opakuje, dokud není vlastnost haldy obnovena nebo dokud nedojde k dosažení kořene stromu. [1]

Při odebrání prvku s nejvyšší prioritou (kořene stromu) se tento prvek odstraní a nahradí se posledním prvkem v haldě. Poté se provádí operace zvaná *heapify-down*. Probíhá porovnávání nově

²Platí pro variantu max-heap, pro min-heap je podmínka opačná.

umístěného prvku s jeho potomky a případné prohození s největším potomkem, pokud je nový prvek menší než některý z jeho potomků. Proces se opět opakuje až do splnění vlastnosti haldy nebo dosažení listu stromu. [1]

Operace vkládání a odebrání prvku s nejvyšší prioritou v binární haldě mají složitost $O(\log n)$, což z ní činí efektivní datovou strukturu pro implementaci prioritních front a pro použití v algoritmech, které vyžadují rychlý přístup k prvkům s nejvyšší prioritou, jako jsou například algoritmy pro hledání nejkratší cesty nebo plánování úloh. Její časté využití je také v třídícím algoritmu Heapsort, který bude detailněji popsán v Kapitole 2.2.2. [2]

2.1.4 Srovnání popsaných datových struktur

Pro přehlednost jsou v této části shrnuty klíčové vlastnosti a časové složitosti základních operací (vyhledávání, vkládání a mazání) u všech výše popsaných datových struktur. Tabulka 2.1 poskytuje srovnání průměrných a nejhorších případů složitosti, které přímo ovlivňují volbu konkrétní struktury pro daný typ úlohy. V posledním sloupci je také uveden hlavní princip, na kterém daná datová struktura funguje, a který je důležitý pro pochopení jejího chování v rámci vizualizace a implementace v aplikaci.

Struktura	Vyhledávání	Vložení/Smazání	Hlavní princip
Spojový seznam	$O(n)$	$O(1) / O(n)$ ³	Dynamické uzly
Zásobník	$O(n)$	$O(1)$	LIFO princip
Fronta	$O(n)$	$O(1)$	FIFO princip
BST	$O(\log n)/O(n)$	$O(\log n)/O(n)$	Uspořádaný strom
AVL strom	$O(\log n)$	$O(\log n)$	Přísná vyváženost
Červeno-černý	$O(\log n)$	$O(\log n)$	Pravidla barev
B-strom	$O(\log n)$	$O(\log n)$	Více klíčů v uzlu
Binární halda	$O(1)$ ⁴	$O(\log n)$	Max/Min vlastnost

Tabulka 2.1: Srovnání vlastností a časové složitosti operací datových struktur

Jak vyplývá z tabulky, zatímco lineární struktury jsou vhodné pro specifické typy přístupu k datům (jako LIFO u zásobníku), u vyhledávacích úloh dominují stromové struktury. Samovyvažující se stromy (AVL a červeno-černý) pak řeší problém degradace BST tím, že garantují logaritmickou složitost i v nejhorším případě. Speciální vlastnosti má B-strom, který díky své šířce optimalizuje přístup k datům v sekundárních úložištích, a binární halda, která je optimalizována pro operace s prioritami.

³Konstantní čas platí pro vkládání/mazání na koncích seznamu (u head/tail).

⁴Namísto hledání konkrétního prvku, což by mělo složitost $O(n)$, je možné získat prvek s nejvyšší prioritou (kořen haldy) v konstantním čase $O(1)$.

Pro účel výukové aplikace zde tedy vznikají tři základní typy struktury, každá s konkrétními operacemi a stylem zobrazení nutným pro jejich názorné představení uživateli.

2.2 Třídící algoritmy

V této kapitole jsou popsány třídící algoritmy, které budou implementovány v aplikaci. Každý algoritmus je charakterizován svým principem fungování, složitostí a významem pro pochopení základních konceptů informatiky.

Většina třídících algoritmů uvedených v této kapitole během práce s polem často využívá vyměňování dvou prvků mezi sebou. Pro zjednodušení implementace a zvýšení přehlednosti kódu je vhodné mít pomocnou funkci pro výměnu dvou prvků v poli, která bude využívána napříč různými algoritmy. Tato funkce má název **Swap**, a její pseudokód je uveden ve Výpisu 1. Kromě této funkce bude také využita funkce **Length**, která vrací délku pole a její implementace je zřejmá a není zde uvedena.

```
1 Function Swap( $A, i, j$ ):  
2    $tmp \leftarrow A[i]$   
3    $A[i] \leftarrow A[j]$   
4    $A[j] \leftarrow tmp$ 
```

Výpis 1: Pseudokód pomocné funkce pro výměnu dvou prvků

Právě funkce **Swap** je klíčová pro vizualizaci třídících algoritmů, protože umožňuje jasně zobrazit, které prvky jsou během třídění vyměňovány. Druhou důležitou operací je porovnávání prvků, které je v pseudokódu reprezentováno pomocí operátorů porovnávání v podmínce (například $<$, $>$, $=$ atd.) a bude také vizualizováno, aby uživatel mohl sledovat, které prvky jsou porovnávány a jak se rozhoduje o jejich výměně. [4]

2.2.1 Algoritmy s kvadratickou časovou složitostí

Jako úvod do problematiky třídících algoritmů jsou nejprve popsány algoritmy s kvadratickou časovou složitostí, které jsou často považovány za základní a slouží jako výchozí bod pro pochopení složitějších algoritmů. Tyto algoritmy jsou charakterizovány tím, že v nejhorším případě provádějí přibližně n^2 operací, což je důvod, proč nejsou vhodné pro třídění velkých množství dat, ale jsou užitečné pro ilustraci základních principů třídění a pro výuku algoritmického myšlení.

V této kapitole budou uvedeny tři základní algoritmy s kvadratickou časovou složitostí: *Bubble Sort*, *Selection Sort* a *Insertion Sort*. Každý z těchto algoritmů bude popsán svým principem fungování, složitostí a případnými optimalizacemi, které mohou být použity pro zlepšení jejich výkonu v určitých situacích.

Bubble sort

Prvním třídícím algoritmem s kvadratickou časovou složitostí, který bude popsán, je *Bubble Sort*. Tento algoritmus funguje na principu opakovaného procházení seznamu, přičemž v každém průchodu nechá nejvyšší prvek „vystoupat“ na konec seznamu, ostatně z analogie k bublinám ve vodě získal svůj název. Algoritmus pokračuje v procházení seznamu, dokud není celý seznam seřazený. [5]

Názorný příklad fungování algoritmu *Bubble Sort* je možné vidět na Výpisu 2, kde jsou zobrazeny jednotlivé kroky algoritmu při třídění pole. V každém kroku jsou porovnány sousední prvky a dle podmínky vyměněny, což vede k postupnému „vystupování“ největších prvků na konec pole.

```
1 Function BubbleSort(A):
2    $n \leftarrow \text{Length}(A)$ 
3   for  $i \leftarrow 0$  to  $n - 1$  do
4     for  $j \leftarrow 0$  to  $n - i - 1$  do
5       if  $A[j] > A[j + 1]$  then
6          $\text{Swap}(A[j], A[j + 1])$ 
```

Výpis 2: Pseudokód algoritmu Bubble Sort

Tento algoritmus může být optimalizován pomocí příznaku, který sleduje, zda došlo k nějaké výměně během průchodu polem. Pokud během průchodu nedojde k žádné výměně, znamená to, že pole je již seřazené, a algoritmus může být ukončen dříve, což zlepšuje jeho výkon v případě, že je pole již téměř seřazené.

I když je tento algoritmus jednoduchý a snadno pochopitelný, jeho kvadratická časová složitost $O(n^2)$ ho činí neefektivním pro třídění velkých množství dat, a proto se v praxi používá jen zřídka, spíše jako pedagogický nástroj pro ilustraci základních principů třídění. Jeho výhodou také může být jeho stabilita a konstantní prostorová složitost $O(1)$, protože třídění probíhá přímo v původním poli bez potřeby dalšího prostoru pro pomocné struktury, dá se tedy využít pro stroje s velmi omezenými zdroji. [5]

Selection sort

Další algoritmus s kvadratickou časovou složitostí je *Selection Sort*, který je často považován za jeden z nejjednodušších třídících algoritmů. V každé iteraci vždy hledá nejmenší prvek v nesetříděné části pole a vymění ho s prvním prvkem této části, čímž postupně rozšiřuje setříděnou část pole. Prvky se tímto způsobem přesouvají, dokud algoritmus nedosáhne konce pole. [5]

Z příkladu na Výpisu 3 je možné vidět, jak algoritmus pracuje. Sekvenčně prochází nesetříděnou část pole, „vybere“ nejmenší prvek a vymění ho s prvním prvkem této části, čímž se rozšiřuje

setříděná část. Ve srovnání s předchozím algoritmem se tedy setříděné pole vytváří zleva, zatímco u Bubble Sortu se největší prvky přesouvají doprava.

```
1 Function SelectionSort(A):
2   n ← Length(A)
3   for i ← 0 to n − 1 do
4     minIndex ← i
5     for j ← i + 1 to n do
6       if A[j] < A[minIndex] then
7         minIndex ← j
8     Swap(A[i], A[minIndex])
```

Výpis 3: Pseudokód algoritmu Selection Sort

Co se týče optimalizace, u algoritmu *Selection Sort* nejsou běžně používané žádné specifické optimalizace, z části i proto, že algoritmus již provádí minimální počet výměn (přesně $n - 1$), což je výhodné v situacích, kdy jsou výměny nákladné. Oproti předchozímu algoritmu Bubble Sort není tento algoritmus stabilní, protože může měnit pořadí prvků se stejnou hodnotou během výměn. Jeho časová i prostorová složitost je stejná jako u Bubble Sort, tedy $O(n^2)$ pro časovou složitost a $O(1)$ pro prostorovou složitost, protože třídění probíhá přímo v původním poli. [5]

Insertion sort

Poslední algoritmus s kvadratickou časovou složitostí, který bude popsán, je *Insertion Sort*. Stejně jako u předchozího algoritmu se zde setříděné pole vytváří zleva, ale místo toho, aby hledal nejmenší prvek v neseřazené části, *Insertion Sort* postupně posune všechny prvky větší než aktuální klíč doprava a vloží klíč na správnou pozici. Tento proces se opakuje pro každý prvek v poli, dokud není celé pole seřazené. [5]

Logiku přesouvání prvků je možné vidět na Výpisu 4, kde se prvně vybere klíč (aktuální prvek) a poté se porovnává s předchozími prvky v setříděné části pole. Pokud je klíč menší než některý z těchto prvků, tyto prvky se posunou doprava, dokud nenajdou správnou pozici pro klíč, který se poté vloží na tuto pozici.

Tento algoritmus má známou mírnou optimalizaci, kde místo toho, aby se klíč s prvky většími než on přehazoval, uloží se jeho hodnota do pomocné proměnné a poté se posouvají pouze větší prvky, dokud není nalezena správná pozice pro klíč, který se poté vloží na tuto pozici. Tato optimalizace snižuje počet výměn, což může zlepšit výkon algoritmu, zejména v případech, kdy je pole již téměř seřazené.

Insertion Sort je stabilní algoritmus, protože nezmění pořadí prvků se stejnou hodnotou během posouvání. Jeho časová složitost je $O(n^2)$ v nejhorsím případě, což nastává, když je pole seřazeno

```

1 Function InsertionSort( $A$ ):
2    $n \leftarrow \text{length}(A)$ 
3   for  $i \leftarrow 1$  to  $n - 1$  do
4      $j \leftarrow i$ 
5     while  $j > 0$  and  $A[j - 1] > A[j]$  do
6       Swap( $A[j - 1], A[j]$ )
7        $j \leftarrow j - 1$ 

```

Výpis 4: Pseudokód algoritmu Insertion Sort

v opačném pořadí, ale v nejlepším případě, kdy je pole již seřazené, má časovou složitost $O(n)$, protože každý prvek je porovnán pouze jednou a není potřeba žádných posunů. Pro prostorovou složitost platí $O(1)$, protože třídění probíhá přímo v původním poli bez potřeby dalšího prostoru pro pomocné struktury. [5]

2.2.2 Algoritmy s logaritmickou časovou složitostí

Po seznámení s algoritmy s kvadratickou časovou složitostí je vhodné přejít k algoritmům, které jsou efektivnější a mají logaritmickou časovou složitost, v nejhorším případě tedy provádějí přibližně $O(n \log n)$ operací. Tyto algoritmy jsou schopné třídit velké množství dat mnohem rychleji než algoritmy s kvadratickou složitostí, a proto jsou často používány v praxi. V této části budou popsány tři základní algoritmy s logaritmickou časovou složitostí: *Merge Sort*, *Quick Sort* a *Heap Sort*. Každý z těchto algoritmů bude charakterizován svým principem fungování, složitostí a případnými optimalizacemi.

Merge sort

Merge Sort je algoritmus založený na principu *divide and conquer*. Algoritmus rekurzivně rozděljuje pole na dvě poloviny, dokud nedosáhne pole o velikosti jednoho prvku, které je považováno za seřazené. Výsledky se poté postupně slučují zpět dohromady, přičemž při slučování se porovnávají prvky z obou polovin a vytváří se nové pole, které je seřazené. Tento proces se provádí rekurzivně a na konci se získá celé pole seřazené. [1]

Princip fungování algoritmu *Merge Sort* je znázorněn na Výpisu 5. Samotná funkce **MergeSort** je velmi jednoduchá, její jediná funkcionalita je nalezení prvku zhruba uprostřed zadaného pole, rekurzivní zavolání sama sebe pro levou a pravou polovinu a následné zavolání funkce **Merge**, která provádí samotné slučování.

```

1 Function Merge( $A, l, m, r$ ):
2    $n_1 \leftarrow m - l + 1$ 
3    $n_2 \leftarrow r - m$ 
4   for  $i \leftarrow 0$  to  $n_1 - 1$  do
5      $L[i] \leftarrow A[l + i]$ 
6   for  $j \leftarrow 0$  to  $n_2 - 1$  do
7      $R[j] \leftarrow A[m + 1 + j]$ 
8    $i \leftarrow 0, j \leftarrow 0, k \leftarrow l$ 
9   while  $i < n_1$  and  $j < n_2$  do
10    if  $L[i] \leq R[j]$  then
11       $A[k] \leftarrow L[i]$ 
12       $i \leftarrow i + 1$ 
13    else
14       $A[k] \leftarrow R[j]$ 
15       $j \leftarrow j + 1$ 
16     $k \leftarrow k + 1$ 
17  while  $i < n_1$  do
18     $A[k] \leftarrow L[i]$ 
19     $i \leftarrow i + 1, k \leftarrow k + 1$ 
20  while  $j < n_2$  do
21     $A[k] \leftarrow R[j]$ 
22     $j \leftarrow j + 1, k \leftarrow k + 1$ 

23 Function MergeSort( $A, l, r$ ):
24   if  $l < r$  then
25      $m \leftarrow l + (r - l)/2$ 
26     MergeSort( $A, l, m$ )
27     MergeSort( $A, m + 1, r$ )
28     Merge( $A, l, m, r$ )

```

Výpis 5: Pseudokód algoritmu Merge Sort

Funkce `Merge` je již delší co se týče implementace. Na začátku překopíruje levou a pravou část pole do dvou pomocných polí a následně u obě z nich prochází sekvenčně zleva a porovnává jejich prvky. Menší prvek z obou polí se vloží do výsledného pole a index se posune o jednu pozici doprava.

Tento proces se opakuje, dokud nejsou všechna data z obou pomocných polí zpracována, přičemž pokud jedno z polí dojde dříve, zbylé prvky z druhého pole se jednoduše překopírují do výsledného pole.

Tento algoritmus má časovou složitost $O(n \log n)$ v nejhorším, průměrném i nejlepším případě, protože vždy rozděluje pole na dvě poloviny a prochází všechny prvky během slučování. Prostorová složitost je však horší než u výše uvedených kvadratických algoritmů, protože vyžaduje dodatečný prostor pro pomocná pole během slučování, což vede k prostorové složitosti $O(n)$. Nicméně, *Merge Sort* je stabilní a velmi dobře se paralelizuje, což z něj činí vhodný algoritmus pro třídění velkých datových sad, zejména v prostředí s více procesory. [5]

Quick sort

Dalším velmi efektivním a často používaným algoritmem, který zde bude zmíněn, je algoritmus *Quick Sort*. Stejně jako *Merge Sort* je založen na principu *divide and conquer*, jeho časová složitost je v průměrném a nejlepším případě $O(n \log n)$, ale v nejhorším případě může dosáhnout až $O(n^2)$. Tento případ je obvykle způsoben nevhodným výběrem pivotu, což je prvek, dle kterého se pole rozděluje na dvě části, jak bude vysvětleno níže. Nicméně, s vhodnými optimalizacemi, jako je například náhodný výběr pivotu, lze dosáhnout průměrné časové složitosti $O(n \log n)$ i v nejhorším případě. [5]

```
1 Function Partition(A, low, high):
2   pivot ← A[high]
3   i ← low − 1
4   for j ← low to high − 1 do
5     if A[j] ≤ pivot then
6       i ← i + 1
7       Swap(A[i], A[j])
8   Swap(A[i + 1], A[high])
9   return i + 1

10 Function QuickSort(A, low, high):
11   if low < high then
12     p ← Partition(A, low, high)
13     QuickSort(A, low, p − 1)
14     QuickSort(A, p + 1, high)
```

Výpis 6: Pseudokód algoritmu Quick Sort

Hlavní funkce tohoto algoritmu je podobná jako v minulém případě. Pokud je pole větší než jeden prvek, zavolá se funkce `Partition`, která rozdělí pole na dvě části podle zvoleného pivotu, jehož index vrací. Následně se funkce `Quick Sort` rekurzivně volá pro obě části pole.

Funkce `Partition` nejprve zvolí pivot, v příkladu na Výpisu 6 se jedná o poslední prvek pole. Poté prochází pole od začátku a porovnává každý prvek s pivotem. Pokud je prvek menší nebo roven pivotu, provede se výměna tohoto prvku s prvkem na indexu i (který začíná na začátku pole) a tento index se posune o jednu pozici doprava. Po dokončení průchodu polem se pivot vymění s prvkem na indexu i , čímž se zajistí, že všechny prvky menší nebo rovné pivotu jsou nalevo od něj a všechny větší prvky jsou napravo. Funkce poté vrací index pivotu, který se používá pro další rekurzivní volání.

Hlavní optimalizace tohoto algoritmu se točí kolem efektivního výběru vhodného pivotu. Pokud totiž je pivot zvolen nevhodně, například jako nejmenší nebo největší prvek v poli, může dojít k tomu, že jedna část pole bude prázdná a druhá bude obsahovat všechny prvky, což vede k nejhoršímu scénáři s časovou složitostí $O(n^2)$. Pro zlepšení výkonu se často používá náhodný výběr pivotu, median-of-three metoda (kde se jako pivot zvolí medián z prvního, prostředního a posledního prvku) nebo jiné heuristiky, které pomáhají zajistit lepší rozdělení pole a tím i lepší výkon algoritmu. [5]

Algoritmus *Quick Sort* je nestabilní, protože může měnit pořadí prvků se stejnou hodnotou během výměn. Jeho prostorová složitost je $O(\log n)$ v průměrném případě, což je způsobeno rekurzivními voláními, ale v nejhorším případě může dosáhnout až $O(n)$, pokud jsou všechny prvky stejné nebo pokud je pole již seřazené a pivot je vždy nejmenší nebo největší prvek. [5]

Heap sort

Jako poslední uvedený algoritmus, který bude v této kapitole popsán, je algoritmus *Heap Sort*. Oproti dvěma předchozím algoritmům je založen na principu *transform and conquer*, protože pracuje s datovou strukturou zvanou halda, která již byla představena v Kapitole 2.1.3 a je možné ji efektivně reprezentovat polem. Algoritmus využívá toho, že přetvoření pole do haldy je možné provést v lineárním čase $O(n)$, a poté opakovaně odebírá prvek s nejvyšší prioritou (kořen haldy) a umísťuje ho na konec pole, čímž postupně vytváří seřazené pole. Druhá část algoritmu má složitost $O(n \log n)$ a díky tomu, že obě části algoritmu jsou prováděny sekvenčně, celková časová složitost algoritmu *Heap Sort* je $\Theta(n \log n)$ v nejhorším, průměrném i nejlepším případě. [1]

Podobně jako u dvou předchozích příkladů se implementace skládá z hlavní funkce `HeapSort`, která je volána na celé pole a která využívá pomocnou funkci `Heapify` pro přetvoření pole do haldy. Nejprve se pole přetvoří do max-heapu, což zajišťuje, že největší prvek je na kořeni haldy. Poté se tento prvek vymění s posledním prvkem pole a z o jeden prvek menšího pole se znovu vytvoří halda.

Funkce `Heapify` je zodpovědná za udržení vlastnosti haldy. Pro každý uzel v poli se porovnává s jeho potomky a pokud je některý z potomků větší než rodičovský uzel, provede se výměna a funkce

```

1 Function Heapify( $A, n, i$ ):
2    $largest \leftarrow i$ 
3    $l \leftarrow 2i + 1$ 
4    $r \leftarrow 2i + 2$ 
5   if  $l < n$  and  $A[l] > A[largest]$  then
6      $largest \leftarrow l$ 
7   if  $r < n$  and  $A[r] > A[largest]$  then
8      $largest \leftarrow r$ 
9   if  $largest \neq i$  then
10    Swap( $A[i], A[largest]$ )
11    Heapify( $A, n, largest$ )

12 Function HeapSort( $A$ ):
13    $n \leftarrow \text{Length}(A)$ 
14   for  $i \leftarrow n/2 - 1$  to 0 do
15     Heapify( $A, n, i$ )
16   for  $i \leftarrow n - 1$  to 1 do
17     Swap( $A[0], A[i]$ )
18     Heapify( $A, i, 0$ )

```

Výpis 7: Pseudokód algoritmu Heap Sort

se rekurzivně volá pro tento potomek, aby se zajistilo, že vlastnost haldy je zachována pro celý strom.

Jak již bylo zmíněno, algoritmus *Heap Sort* má časovou složitost $\Theta(n \log n)$ v nejhorším, průměrném i nejlepším případě, protože přetvoření pole do haldy je lineární operace a následné odebírání prvku s nejvyšší prioritou a udržování vlastnosti haldy má logaritmickou složitost. Prostorová složitost je $O(1)$, protože třídění probíhá přímo v původním poli bez potřeby dalšího prostoru pro pomocné struktury. Algoritmus *Heap Sort* je nestabilní, protože může měnit pořadí prvků se stejnou hodnotou během výměn. [1]

2.2.3 Srovnání třídících algoritmů

Pro usnadnění výběru vhodného algoritmu a pro přehlednost teoretických poznatků jsou v této kapitole shrnuty klíčové vlastnosti všech implementovaných algoritmů. Srovnání se zaměřuje na časovou složitost v různých případech, prostorovou náročnost a stabilitu, což je schopnost algoritmu zachovat původní relativní pořadí prvků se stejným klíčem.

Algoritmus	Časová složitost		Prostorová složitost	Stabilita
	Průměrná	Nejhorší		
Bubble Sort	$O(n^2)$	$O(n^2)$	$O(1)$	Ano
Selection Sort	$O(n^2)$	$O(n^2)$	$O(1)$	Ne
Insertion Sort	$O(n^2)$	$O(n^2)$	$O(1)$	Ano
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n)$	Ano
Quick Sort	$O(n \log n)$	$O(n^2)$	$O(n)$	Ne
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(1)$	Ne

Tabulka 2.2: Srovnání vlastností a složitosti třídících algoritmů

Z Tabulky 2.2 je patrné, že algoritmy s kvadratickou časovou složitostí jsou vhodné spíše pro menší datové sady nebo pro specifické případy, jako je téměř seřazené pole u algoritmu *Insertion Sort*, kde jeho nejlepší časová složitost dosahuje lineárních hodnot $O(n)$.

Naopak algoritmy s logaritmickou složitostí vykazují výrazně vyšší efektivitu u velkých datových sad. *Merge Sort* sice vykazuje vyšší prostorovou náročnost $O(n)$, ale na rozdíl od *Quick Sortu* a *Heap Sortu* garantuje stabilitu třídění. *Quick Sort* je v praxi často nejrychlejší, avšak jeho výkon je silně závislý na volbě pivotu, což se projevuje v riziku nejhoršího případu $O(n^2)$. *Heap Sort* představuje vyvážený kompromis, který garantuje logaritmický čas i v nejhorším případě při zachování konstantní prostorové složitosti.

Výše uvedené algoritmy byly tedy označeny jako klíčové pro implementaci v rámci výukové aplikace, protože pokrývají široké spektrum třídících technik a umožňují uživatelům pochopit různé přístupy k třídění dat, jejich výhody a nevýhody.

Kapitola 3

Analýza existujících řešení

V této kapitole jsou popsány vybrané existující aplikace, které slouží k vizualizaci algoritmů a datových struktur. Cílem je analyzovat jejich funkce, uživatelské rozhraní a možnosti interakce, aby bylo možné identifikovat silné a slabé stránky těchto nástrojů. Tato analýza poslouží jako základ pro návrh a implementaci nového interaktivního nástroje, který bude lépe vyhovovat potřebám uživatelů.

Analýza se nejprve zaměří na vizualizaci datových struktur a následně na simulaci třídících algoritmů. Vybrané aplikace budou hodnoceny z hlediska jejich uživatelské přívětivosti, rozsahu podporovaných algoritmů a datových struktur, možnosti interakce a přizpůsobení vizualizací.

3.1 Vizualizace datových struktur

Nástroje popsané v této kapitole se zaměřují na vizualizaci různých datových struktur, jako jsou stromy, grafy, zásobníky a fronty.

3.1.1 Gnarley Trees

První aplikací, kterou tato práce analyzuje, je *Gnarley Trees*, interaktivní nástroj pro vizualizaci a manipulaci s různými typy stromových datových struktur. [6]

Obsah

Aplikace se zaměřuje především na stromy a haldy, v kontextu různých způsobů ukládání dat a jejich efektivity. U stromů jde například o binární vyhledávací stromy, červeno-černé stromy, AVL stromy, B-stromy a různé další varianty těchto struktur. U front a hald jsou to například prioritní fronty implementované pomocí běžných hald, binárních hald, Fibonacciho hald a dalších.

Každý typ datové struktury má možnost vizualizace základních operací, u stromů například vkládání, mazání a vyhledávání uzlů, u hald a front pak vkládání, inkrementaci hodnoty prvku a

odstraňování prvků s nejvyšší prioritou. Dále je zde také možnost vložit předem definovaný počet náhodných prvků a tím rychle zaplnit danou strukturu. Tyto interakce jsou prováděny pomocí uživatelského rozhraní, které umožňuje uživatelům zadávat hodnoty a sledovat, jak se datová struktura mění v reálném čase. Vizualizace je doplněna o animace, které umožňují mimo jiné i krokování operací, což usnadňuje pochopení dynamiky datových struktur. Animace jsou jednoduché, ale efektivní, včetně možnosti krokování vpřed i vzad.

Vizuál

Jedná se o webovou aplikaci napsanou v jazyce Java, vykreslování grafů je realizováno pomocí knihovny CheerpJ. Uživatelské rozhraní je přehledné, umožňuje snadný přístup k různým datovým strukturám a jejich operacím. Kromě toho aplikace ke všemu přikládá vysvětlivky, což je užitečné pro uživatele, kteří se s těmito koncepty teprve seznamují.

Co se týče uživatelského přizpůsobení, stránka má pouze základní stylování, bez možnosti výběru barevných schémat nebo přizpůsobení vzhledu. Stránka také zřejmě má možnost změny jazyka z angličtiny do slovenštiny, ale její zapnutí není na první pohled zřejmé.

3.1.2 Data Structure Visualizations

Další aplikací je *Data Structure Visualizations*, výukový nástroj z Univerzity San Francisco, který nabízí vizualizace různých datových struktur a algoritmů. [7]

Obsah

Aplikace pokrývá širokou škálu témat, jak v oblasti datových struktur, tak i algoritmů. Mezi datové struktury patří například zásobníky, fronty, spojové seznamy, stromy (včetně binárních vyhledávacích stromů a hald) a grafy. U každé datové struktury jsou k dispozici vizualizace základních operací, jako je vkládání, mazání a vyhledávání prvků. U algoritmů jsou zahrnuty různé třídící algoritmy (například Bubble Sort, Insertion Sort, Merge Sort) a algoritmy pro prohledávání grafů (například Dijkstrův algoritmus nebo algoritmy na získání kostry grafu).

Interakce s vizualizacemi není sjednocená jako u minulé aplikace, každá datová struktura ji má implementovanou jinak. Animace pro jednotlivé operace jsou velmi jednoduché a oproti předchozí aplikaci postrádají vysvětlivky. Krokování funguje opět vpřed i vzad, včetně možnosti nastavení rychlosti animace. Jako nedostatek lze uvést i absenci možnosti naplnění datové struktury náhodnými hodnotami, což by zrychlilo testování a experimentování s různými scénáři. Třídící algoritmy jsou velmi jednoduché, umožňují uživateli náhodně zamíchat pole čísel, zvolit si třídící algoritmus a sledovat jeho průběh. Jako velmi zajímavé rozšíření aplikace poskytuje možnost napsat si vlastní algoritmus v jazyce JavaScript a vizualizovat jeho průběh.

Vizuál

Aplikace je webová, napsaná v jazyce JavaScript, s jednoduchým uživatelským rozhraním. Celkový vzhled není příliš moderní a uživatelsky přívětivý, ale funkční. Navigace na stránce je mírně komplikovaná, protože uživatel musí procházet různými odkazy, aby se dostal k požadované datové struktuře nebo algoritmu. Ovládání vizualizací je stylováno pomocí tlačítek a vstupních polí, ve stejném duchu jako předchozí aplikace.

Možnosti přizpůsobení jsou omezené, bez možnosti změny barevných schémat nebo vzhledu vizualizací. Responzivita stránky je značně omezená, což může ztížit použití na různých zařízeních. Aplikace je dostupná pouze v angličtině.

3.1.3 B Plus Tree

Poslední aplikací zaměřující se na datové struktury, jejíž analýzu tato práce provede, je *B Plus Tree*, interaktivní nástroj pro vizualizaci B+ stromů. [8]

Obsah

Aplikace se specializuje výhradně na B+ stromy, neobsahuje tedy jiné datové struktury. U B+ stromů jsou k dispozici vizualizace základních operací, jako je vkládání, mazání a vyhledávání klíčů. Mimo tyto poskytuje i možnost nastavit danému stromu maximální počet klíčů na vnitřní uzel nebo list a také možnost generování náhodných klíčů pro rychlé naplnění stromu. Chybí zde vysvětlivky k jednotlivým operacím, krátký popis struktury se věnuje spíše využití B+ stromů v databázových systémech.

Interakce s vizualizacemi je intuitivní, uživatel může zadávat hodnoty prostřednictvím vstupních polí a sledovat, jak se strom mění v reálném čase. Animace nemají možnost krokování, po provedení operace se strom aktualizuje rovnou do nového stavu, a také je není možné krokovat zpět.

Vizuál

Tato aplikace je rovněž webová, s velmi jednoduchým ale moderním uživatelským rozhraním. Obsah je tak malý, že není potřeba složitější navigace, vše je dostupné přímo na hlavní stránce. Ovládání vizualizací je realizováno pomocí tlačítek a vstupních polí. Stránka nenabízí žádné možnosti přizpůsobení vzhledu nebo změnu jazyka. Aplikace je velmi responzivní a funguje dobře na různých zařízeních.

3.1.4 Porovnání

V této kapitole jsou shrnuty klady a zápory jednotlivých analyzovaných aplikací zaměřených na vizualizaci datových struktur. Následující tabulka (Tabulka 3.1) poskytuje přehledné srovnání jejich vlastností.

Vlastnost	Gnarley Trees	Data Structure Visualizations	B Plus Tree
Podpora více datových struktur	Ano	Ano	Ne
Uživatelské přizpůsobení	Omezené	Omezené	Žádné
Krokování animací	Ano (vpřed i vzad)	Ano (vpřed i vzad)	Ne
Vysvětlivky k operacím	Ano	Ne	Ne
Generování náhodných dat	Ano	Ne	Ano
Responzivita	Dobrá	Slabá	Výborná
Jazyková podpora	Angličtina, slovenština	Angličtina	Angličtina

Tabulka 3.1: Porovnání analyzovaných aplikací pro vizualizaci datových struktur

Dále byla provedena analýza a porovnání konkrétních datových struktur, které výše zmíněné aplikace podporují. Tabulka 3.2 shrnuje, které datové struktury jsou v jednotlivých aplikacích podporovány.

Datová struktura	Gnarley Trees	Data Structure Visualizations	B Plus Tree
Binární vyhledávací strom	Ano	Ano	Ne
AVL strom	Ano	Ano	Ne
Červeno-černý strom	Ano	Ano	Ne
B-strom	Ano	Ano	Ne
B+ strom	Ne	Ano	Ano
Spojový seznam	Ano ¹	Ne	Ne
Zásobník	Ano ¹	Ano	Ne
Fronta	Ano ¹	Ano	Ne
Halda	Ano	Ano	Ne

Tabulka 3.2: Podporované datové struktury v analyzovaných aplikacích

Z uvedeného shrnutí je patrné, že nejkomplexnější podporu datových struktur nabízí aplikace *Data Structure Visualizations*, která pokrývá téměř všechny uvedené struktury. *Gnarley Trees* nabízí také širokou škálu datových struktur, ale některé z nich jsou dostupné pouze v teoretické části aplikace a nejsou vizualizovány. Nejméně obsáhlá je aplikace *B Plus Tree*, která se sice specializuje pouze na B+ stromy, ale zato je v této oblasti velmi dobře propracovaná.

3.2 Simulace třídících algoritmů

Tato kapitola se věnuje aplikacím, které umožňují vizualizaci a simulaci různých třídících algoritmů, jako jsou Bubble Sort, Quick Sort, Merge Sort a další.

¹Pouze v teoretické části.

3.2.1 Sort Visualizer

Aplikace *Sort Visualizer* je interaktivní nástroj pro vizualizaci různých třídících algoritmů, který umožňuje uživatelům sledovat průběh třídění v reálném čase spolu s popisem algoritmů a jejich časovou a prostorovou složitostí. [9]

Obsah

Sort Visualizer nabízí simulace několika běžných třídících algoritmů, které dělí na logaritmické (Quick Sort, Merge Sort, Heap Sort), kvadratické (Bubble Sort, Insertion Sort, Selection Sort) a další (např. Radix Sort). Přidává také možnost, jak si uživatel může vytvořit vlastní algoritmus pomocí jazyka JavaScript a vizualizovat jeho průběh.

Interakce se simulací je značně omezená, uživatel si může pouze zvolit algoritmus, velikost pole a rychlost animace. Animace spočívá v zobrazení sloupců reprezentujících hodnoty v poli, které se mění podle průběhu třídění. Zajímavé zpestření animace jsou zvukové efekty, které doprovázejí porovnávání a výměny prvků a svým tónem reprezentují hodnotu daného prvku. Krokování animace není možné.

Každý algoritmus je doplněn o krátký popis jeho principu, časové složitosti v nejlepším, průměrném a nejhorším případě a také o prostorovou složitost. Přiloženy jsou také zdrojové kódy daného algoritmu v několika programovacích jazycích. Zdrojové kódy jsou však statické, nelze je během simulace nijak měnit a také nejsou zvýrazněny části kódu, které se právě vykonávají.

Vizuál

Tato webová aplikace je napsaná pomocí webového frameworku Flask, s moderním a přehledným uživatelským rozhraním. Navigace na stránce je jednoduchá, všechny třídící algoritmy jsou vždy dostupné z navigačního panelu. Samotná vizualizace je vizuálně velmi poutavá a v doprovodu se zvukovými efekty působí přívětivě.

Aplikace nenabízí žádné možnosti přizpůsobení vzhledu nebo změnu jazyka. Výchozí barevné schéma je v temném režimu. Responzivita stránky je skvělá, stránka funguje dobře téměř v jakémkoliv rozlišení.

3.2.2 VisuAlgo

Jedná se o další webovou aplikaci, sloužící jako výukový nástroj pro vizualizaci všemožných algoritmů a datových struktur, která byla vyvinuta na National University of Singapore. Aplikace *VisuAlgo* nabízí širokou škálu vizualizací, včetně třídících algoritmů. [10]

Obsah

Rozsah této aplikace je velmi široký, pokrývá mnoho témat v oblasti algoritmů a datových struktur. Stránka nabízí vizualizace pro různé datové struktury, jako jsou zásobníky, fronty, spojové seznamy, stromy a grafy. Kromě toho obsahuje i vizualizace různých algoritmů, včetně třídících algoritmů, algoritmů pro prohledávání grafů a dalších. Obsahuje i interaktivní cvičení a kvízy, které pomáhají uživatelům lépe pochopit koncepty.

V kontextu třídících algoritmů nabízí vizualizace pro algoritmy jako Bubble Sort, Insertion Sort, Selection Sort, Merge Sort, Quick Sort a Heap Sort. Animace jsou jednoduše znázorněny opět pomocí sloupců reprezentujících hodnoty v poli. Uživatel má možnosti nastavení rychlosti přehrávání, plné krokování animace vpřed i vzad a také možnost pozastavení animace. Kromě toho aplikace poskytuje i podrobné vysvětlení principu každého algoritmu, včetně jeho časové a prostorové složitosti. Během animace je přístupný i pseudokód algoritmu, který je zvýrazňován podle aktuálně vykonávané části.

Vizuál

Webové stránky mají značně zaplněné uživatelské rozhraní, což může být pro nové uživatele trochu matoucí. Navigace na stránce je relativně dobře organizovaná, s jasnými odkazy na různé sekce a témata. Stránka s vizualizacemi se může zdát přeplněná ještě více, protože obsahuje mnoho ovládacích prvků a informací. Toto okno také obsahuje vysvětlivky, které jsou provedeny ve vyskakovacích oknech, které uživatele zahltní při prvním použití.

Výchozí barevné schéma je světlé, uživatel si nemůže změnit režim na tmavý. Responzivita stránky je dobrá, když se rozlišení zmenší na nedostatečnou šířku, aplikace uživatele upozorní, že vizualizace nemusí fungovat správně. Aplikace podporuje změnu jazyka, na výběr uživatelé mají z angličtiny, čínštiny a indonéštiny.

3.2.3 Toptal Sorting Algorithms Animations

Poslední analyzovanou aplikací je *Sorting Algorithms Animations* od společnosti Toptal, téměř jednostránkový webový nástroj pro vizualizaci a porovnání různých třídících algoritmů. [11]

Obsah

Aplikace nabízí vizualizace pro několik běžných třídících algoritmů, které již byly zmíněny výše. Jednotlivé algoritmy jsou umístěny v matici, kde každý sloupec reprezentuje jeden algoritmus a každý řádek představuje typ vstupních dat (např. náhodná data, téměř seřazená data nebo obráceně seřazená data). Pomocí tohoto rozložení může uživatel snadno porovnat výkon různých algoritmů na různých typech vstupů.

Interakce s vizualizacemi je velmi omezená, uživatel si může pouze zvolit algoritmus a typ vstupních dat. Animace jsou jednoduché, zobrazují sloupce reprezentující hodnoty v poli, které se mění podle průběhu třídění. Krokování animace není možné.

Uživatel má také možnost zobrazit si detail konkrétního algoritmu, kde je k němu přiložena vysvětlivka, pseudokód a časová a prostorová složitost. Rozkliknutí konkrétního datového typu zase zobrazí konkrétní porovnání časové složitosti všech algoritmů na daném typu vstupu a výčet poznatků o jejich výkonu.

Vizuál

Jedná se o velmi jednoduchou webovou aplikaci. Hlavní stránka obsahuje pouze výše uvedenou matici algoritmů a typů vstupních dat spolu s obecnou diskuzí na téma třídících algoritmů. Detail algoritmu je veden ve stejném stylu, animace jsou pouze větší a jsou doplněny o pseudokód a diskuzi ke konkrétnímu algoritmu.

Stránka nemá žádné možnosti přizpůsobení vzhledu nebo změnu jazyka. Výchozí barevné schéma je světlé. Jako jediná z dosud analyzovaných aplikací také obsahuje reklamy. Responzivita stránky je dobrá, funguje dobře na různých zařízeních.

3.2.4 Porovnání

V této kapitole jsou shrnuty klady a zápory jednotlivých analyzovaných aplikací zaměřených na vizualizaci třídících algoritmů. Následující tabulka (Tabulka 3.3) poskytuje přehledné srovnání jejich vlastností.

Vlastnost	Sort Visualizer	VisuAlgo	Sorting Algorithms Animations
Uživatelské přizpůsobení	Žádné	Žádné	Žádné
Krokování animací	Ne	Ano (vpřed i vzad)	Ne
Vysvětlivky k operacím	Ano	Ano	Ano
Zvukové efekty	Ano	Ne	Ne
Responzivita	Výborná	Dobrá	Dobrá
Jazyková podpora	Angličtina	Angličtina, čínština, indonéština	Angličtina

Tabulka 3.3: Porovnání analyzovaných aplikací pro vizualizaci třídících algoritmů

Z konkrétní analýzy vyšlo najevo, že všechny tři aplikace podporují základní množinu nejznámějších třídících algoritmů, ale liší se v možnostech interakce a uživatelského přizpůsobení. *VisuAlgo* nabízí nejvíce možností interakce, včetně krokování animací a podrobného pseudokódu, zatímco *Sort Visualizer* a *Topical Sorting Algorithms Animations* mají omezené možnosti interakce. Zvukové efekty jsou unikátní funkcí *Sort Visualizer*, která přidává další vrstvu zážitku z vizualizace.

3.3 Shrnutí

Analýza existujících řešení, provedená v této kapitole, poskytla cenné poznatky o současných nástrojích pro vizualizaci datových struktur a třídících algoritmů. Z analýzy vyplynulo, že zatímco některé aplikace nabízejí širokou škálu funkcí a interakcí, jiné se zaměřují na specifické datové struktury nebo algoritmy s omezenými možnostmi přizpůsobení.

Na základě porovnání podporovaných datových struktur bylo rozhodnuto, že modelová aplikace bude zahrnovat vizualizaci následujících datových struktur: binární vyhledávací strom, AVL strom, červeno-černý strom, B-strom, B+ strom, spojový seznam, zásobník, fronta a halda. Tato volba byla učiněna s ohledem na jejich význam v oblasti informatiky a jejich časté využití v praxi.

Co se týče třídících algoritmů, modelová aplikace bude podporovat vizualizaci následujících algoritmů: Bubble Sort, Insertion Sort, Selection Sort, Merge Sort, Quick Sort a Heap Sort. Tyto algoritmy byly vybrány na základě jejich rozšířenosti a různorodosti přístupů k třídění dat.

Zdůrazněna byla také potřeba uživatelsky přívětivého rozhraní, které umožní snadnou navigaci a interakci s vizualizacemi. Dále bylo identifikováno několik klíčových funkcí, které by měly být zahrnuty do modelové aplikace, včetně možnosti krokování animací, generování náhodných dat a poskytování vysvětlivek k jednotlivým operacím.

Kapitola 4

Požadavky na výukovou aplikaci

Pro úspěšný vývoj a implementaci webové aplikace pro vizualizaci datových struktur a algoritmů je nezbytné stanovit jasné požadavky, které budou sloužit jako vodítko během celého vývojového procesu. Jako inspirace pro tyto požadavky byly použity existující nástroje popsané v Kapitole 3. Požadavky jsou rozděleny do několika klíčových kategorií, zahrnujících funkcionalitu, použitelnost, spolehlivost, výkonnost a podporovatelnost a jsou popsány v následujících kapitolách.

4.1 Funkcionalita

Tato kapitola definuje klíčové funkční požadavky na systém, které zahrnují vizualizační schopnosti, interaktivní prvky a vzdělávací obsah.

1. Moduly pro vizualizaci datových struktur

- Implementace a vizualizace struktur: zásobník, fronta, lineární seznamy, pole a stromy.
- Grafická simulace standardních operací nad těmito strukturami.

2. Moduly pro simulaci algoritmů

- Algoritmy řazení: Bubble sort, Merge sort, Quick sort a další.
- Algoritmy vyhledávání: lineární a binární vyhledávání.

3. Interaktivní ovládání simulací

- Funkce krokování algoritmu (vpřed i vzad) pro detailní analýzu průběhu.
- Možnost modifikace vstupních dat v reálném čase s okamžitým promítnutím do vizualizace.

4. Teoretický a metodický podklad

- Formální vysvětlení principů a typických scénářů užití u každé struktury a algoritmu.
- Specifikace časové a prostorové algoritmické složitosti.
- Zobrazení odpovídajícího pseudokódu pro lepší pochopení implementace.

5. Lokalizace rozhraní

- Plná podpora vícejazyčného rozhraní, minimálně v rozsahu českého a anglického jazyka.

6. Architektura na straně klienta

- Provoz aplikace v rámci webového prohlížeče bez nutnosti instalace dodatečných doplňků.
- Provádění veškerých simulací a interakcí lokálně na zařízení uživatele.
- Persistence uživatelského nastavení a historie prostřednictvím webového úložiště (local storage).

4.2 Použitelnost

Cílem požadavků na použitelnost je zajistit nízkou bariéru vstupu a intuitivní ovládání pro cílovou skupinu studentů.

1. Ergonomie a ovládání

- Intuitivní uživatelské rozhraní doplněné o kontextové nápovědy pro složitější ovládací prvky.
- Konzistentní navigační systém pro přehledný přechod mezi moduly struktur a algoritmů.

2. Didaktická podpora

- Synergie vizualizace, teorie a vysvětlujících textů pro podporu uživatelů s různou úrovní vstupních znalostí.

3. Vizuální prezentace

- Implementace plynulých animací pro jasnou identifikaci změn stavu datových struktur.
- Moderní, přehledný a čistý design uživatelského rozhraní odpovídající současným standardům (UI/UX).

4.3 Spolehlivost

Požadavky na spolehlivost se zaměřují na stabilitu systému a korektní zpracování uživatelských dat.

1. Stabilita systému

- Zajištění plné funkčnosti a stability při zpracování datových sad o rozsahu až 100 prvků.
- Validace a ošetření všech neplatných uživatelských vstupů před jejich zpracováním.

2. Odolnost vůči chybám

- Implementace informativních chybových hlášení, která uživatele srozumitelně navedou k nápravě při nekorektní operaci.

4.4 Výkonnost

Klíčovým aspektem výkonnosti je plynulost grafického výstupu a rychlá odezva na interakci.

1. Optimalizace vykreslování

- Zajištění konstantní plynulosti animací nezávisle na zvolené velikosti vstupních dat (v rámci stanovených limitů).

2. Efektivita výpočtů

- Optimalizované zpracování algoritmů na straně klienta pro minimalizaci latence mezi uživatelským vstupem a vizuální odezvou.

4.5 Podpořitelnost a udržitelnost

Požadavky definují technický rámec pro budoucí rozšiřování a údržbu aplikace.

1. Modularita a separace zájmů

- Striktní oddělení vizualizační vrstvy od aplikační logiky.
- Definice jasného a jednoduchého rozhraní (API) mezi jednotlivými komponentami systému.
- Architektura UI umožňující snadné úpravy vzhledu bez zásahu do logiky (např. pomocí CSS proměnných).

2. Technologický zásobník

- Vývoj klientské části s využitím komponentního frameworku Svelte.
- Využití specializovaných knihoven pro vizualizaci dat (např. D3.js nebo vis.js).

3. Dokumentace a rozšiřitelnost

- Detailní dokumentace systémové architektury pro usnadnění budoucího vývoje.
- Návrh systému umožňující snadné přidávání nových datových struktur, algoritmů nebo nových podporovaných jazyků.

Kapitola 5

Analýza a volba použitých technologií

Pro úspěšnou realizaci projektu je prvně nutné vhodně zvolit technologie, které budou použity v jeho implementaci. Tato analýza je provedena v této kapitole, nejprve **TODO: napsat tohle**.

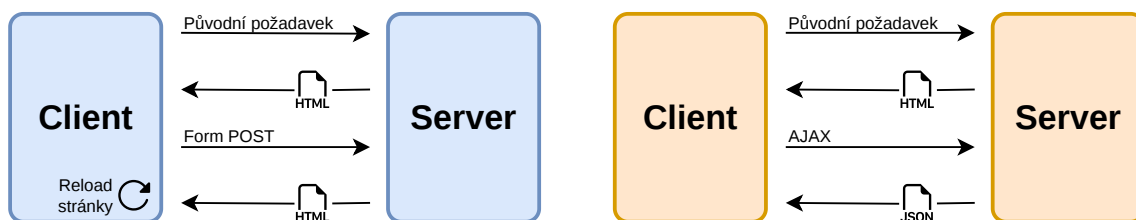
5.1 Architektura aplikace

Ještě před samotnou volbou vhodného frameworku je důležité provést krátkou analýzu možné architektury pro výukovou aplikaci. Architektura určuje, kde bude implementována logika aplikace, jak budou zpracovávány požadavky uživatele a jakým způsobem se bude provádět zobrazování stránky.

5.1.1 Client-Server a Jednostránková webová aplikace

Prvním možným přístupem je klasická **Client-Server** architektura, kdy je logika aplikace implementována na serveru a klient (prohlížeč) pouze zobrazuje výsledky, přičemž komunikace mezi nimi probíhá pomocí HTTP požadavků. Příklad fungování této architektury je zobrazen na Obrázku 5.1, konkrétně jeho levé, modré části. Interakce uživatele s aplikací funguje tak, že při každé akci uživatele (například kliknutí na tlačítko) je odeslán HTTP požadavek na server, který zpracuje požadavek, provede potřebné operace a jako odpověď vrátí novou HTML stránku, která se následně načte v prohlížeči. V praxi to tedy znamená nutnost znovunačtení stránky s každou takovou interakcí. Tento přístup je poměrně jednoduchý a dobře známý, ale může být pomalý a neefektivní, zejména pokud se jedná o interaktivní aplikaci s častými aktualizacemi a obzvláště když stránka nepotřebuje uchovávat stav nebo jiné informace přímo na serveru.

Jako reakce na nevýhody klasické architektury se v posledních letech rozšířil přístup zvaný **Jednostránková webová aplikace** (*Single Page Application*, dále *SPA*). Hlavním rozdílem mezi SPA a klasickou architekturou je to, že v případě SPA jsou data získána hned při prvním požadavku u načtení stránky a následně se již neodesílají další požadavky o další stránky. Namísto toho se veškerá logika a zpracování provádí na straně klienta, přičemž komunikace se serverem probíhá pouze pro získávání dat (například pomocí API). Jak je vidět na pravé, oranžové části Obrázku 5.1,



Obrázek 5.1: Srovnání životních cyklů stránek v klasické client-server architektuře a v SPA

využívá se zde technologie *AJAX* a odpověď ze serveru jsou tentokrát data ve formátu *JSON*, která jsou následně zpracována a zobrazena na stejné stránce bez nutnosti jejího obnovení. [12]

Výhodou tohoto přístupu je rychlejší a plynulejší uživatelský zážitek, protože není nutné znovu načítat celou stránku s každou interakcí. Navíc umožňuje lepší využití zdrojů a efektivnější komunikaci se serverem, protože se odesílají pouze potřebná data namísto celé stránky. SPA také může být výhodná v případě, že uživatel nemá vždy stabilní připojení k internetu, protože většina logiky a zpracování se provádí na klientovi, což umožňuje aplikaci fungovat i v offline režimu, pokud již předtím načetla potřebná data.

Nevýhodou však může být složitější vývoj a údržba, protože veškerá logika a zpracování musí být implementována na straně klienta, což může být náročnější než implementace na serveru. Problémy také nastávají s optimalizací pro vyhledávače (SEO), protože obsah není vždy dostupný pro indexování, a také s bezpečností, protože veškerá logika je přístupná na straně klienta.

Z důvodu výše uvedených výhod a nevýhod byla pro tento projekt zvolena architektura typu SPA, která je vhodnější pro interaktivní aplikace s častými aktualizacemi a umožňuje lepší uživatelský zážitek bez nutnosti časté komunikace se serverem.

5.1.2 Strategie renderování

TODO: pokračovat tady, SSR vs CSR [13]

5.2 Webový framework

5.3 Grafová knihovna

5.4 CI/CD

5.5 Shrnutí

Kapitola 6

Závěr

Tady končíme, všichni ven.

Literatura

1. LEVITIN, Anany. *The Design & Analysis of Algorithms*. 2nd ed. Boston: Pearson Addison-Wesley, 2007. ISBN 978-0-321-36413-5.
2. CORMEN, Thomas H.; LEISERSON, Charles Eric; RIVEST, Ronald L.; STEIN, Clifford. *Introduction to algorithms*. 2nd ed. Boston: McGraw-Hill Book Company, 2001. ISBN 978-0262032933.
3. *Insertion in Red-Black Tree* [online]. GeeksForGeeks, 2025. [cit. 2026-02-08]. Dostupné z: <https://www.geeksforgeeks.org/dsa/insertion-in-red-black-tree/>.
4. SEDGEWICK, Robert; WAYNE, Kevin Daniel. *Algorithms*. Fourth edition. Boston: Addison-Wesley, 2011. ISBN 978-0-321-57351-3.
5. *Sorting Algorithms Explained* [online]. FreeCodeCamp, 2019. [cit. 2026-02-16]. Dostupné z: <https://www.freecodecamp.org/news/sorting-algorithms-explained-with-examples-in-python-java-and-c/>.
6. KOVÁČ, Jakub. *Gnarley Trees* [online]. 2011. [cit. 2026-01-31]. Dostupné z: <https://kubokovac.eu/gnarley-trees/>.
7. GALLES, David. *Data Structure Visualizations* [online]. University of San Francisco, 2011. [cit. 2026-01-31]. Dostupné z: <https://www.cs.usfca.edu/~galles/visualization/>.
8. DICKEN, Ben. *B plus tree* [online]. PlanetScale, 2024. [cit. 2026-01-31]. Dostupné z: <https://bplustree.app/>.
9. SCANU, Daniel. *Sort Visualizer* [online]. 2021. [cit. 2026-01-31]. Dostupné z: <https://www.sortvisualizer.com/>.
10. HALIM, Steven. *VisuAlgo* [online]. National University of Singapore, 2011. [cit. 2026-01-31]. Dostupné z: <https://visualgo.net/en>.
11. *Sorting Algorithms Animations* [online]. Toptal, 2010. [cit. 2026-01-31]. Dostupné z: <https://www.toptal.com/developers/sorting-algorithms>.
12. PANCHAL, Kapil. *SPA in ASP.NET Core* [online]. iFour Technolab, 2020. [cit. 2026-03-07]. Dostupné z: <https://www.ifourtechnolab.com/blog/spa-in-asp-net-core>.

13. OSMANI, Addy; MILLER, Jason. *Rendering on the Web* [online]. web.dev, 2019. [cit. 2026-03-07]. Dostupné z: <https://web.dev/articles/rendering-on-the-web>.